CHAPTER 1

INTRODUCTION

1.1 Overview

Autism Spectrum Disorder (ASD) is a complex neurodevelopmental condition characterized by persistent deficits in social communication and interaction, along with restricted, repetitive patterns of behavior, interests, or activities. The term 'spectrum' reflects the wide variation in challenges and strengths possessed by each person with autism. While some individuals with ASD may be highly skilled in visual, musical, and academic domains, others may face significant cognitive and sensory impairments. Diagnosis typically occurs in early childhood, though many are diagnosed much later, especially in regions where awareness and resources are limited.

The World Health Organization estimates that 1 in 100 children globally has ASD, highlighting the necessity for robust diagnostic tools and interventions. ASD's early symptoms usually appear by age two or three, and early intervention often yields the best results. However, due to the lack of universal screening programs and the limited availability of specialized clinicians, many individuals go undiagnosed or receive a delayed diagnosis, adversely affecting their developmental outcomes.

Advancements in neuroscience, neuroimaging technologies, and computational models have introduced new possibilities for ASD research. Functional Magnetic Resonance Imaging (fMRI) and Electroencephalography (EEG) are among the tools used to analyze the brain's structure and activity. Yet, translating complex brain imaging data into accurate diagnostic conclusions remains a major challenge due to the brain's intricate connectivity and variability across individuals. Traditional approaches have had limited success in incorporating this complexity.

Artificial Intelligence (AI), and more specifically Deep Learning (DL), has demonstrated tremendous potential in the medical field. These models can identify patterns in data that may be imperceptible to human observers. Within this context, Graph Neural Networks (GNNs) have emerged as powerful tools capable of modeling relational data. The human brain, often

conceptualized as a network of interconnected regions, fits naturally into the graph-based paradigm. This compatibility has inspired the use of GNNs for analyzing neuroimaging data to support ASD diagnosis.

NeuroGraph is one such model that applies the GNN framework to capture both the spatial and functional relationships between different regions of the brain. By modeling each individual's brain as a graph, NeuroGraph can identify characteristic patterns associated with ASD and differentiate between affected and non-affected individuals. This method represents a shift toward interpretable, data-driven diagnostics with the potential to augment clinical decision-making.

1.2 Existing System and Drawbacks

The gold standard for ASD diagnosis at present is based on behavioral measures, specifically the Autism Diagnostic Observation Schedule (ADOS) and the Autism Diagnostic Interview-Revised (ADI-R). These have been comprehensively validated and are accepted as highly credible measures across research and clinical communities. Their operational use, however, poses difficulties. They take a lot of training and skill to give and to score correctly, which restricts their availability—most importantly, in settings where resources are limited and trained clinicians are not readily available. Additionally, even though they are standardized in format, they are subjective by nature, heavily dependent on clinician observation and judgment. Subjectivity brings inconsistency in diagnoses due to factors like the clinician's experience, cultural background, and situational factors.

As a countermeasure to such limitations, machine learning (ML) methods have in recent years come into play to aid or augment ASD diagnosis, especially by examining neuroimaging data. Such computational models base their decisions on features derived from brain scans—e.g., gray matter volume, functional connectivity strength, or patterns of activation—to identify patterns indicating ASD. Typically used models are support vector machines (SVMs), random forests, and more currently, convolutional neural networks (CNNs). These models have been able to achieve high classification accuracy and also have the potential to detect subtle neural markers that cannot be observed using behavioral evaluations.

Yet, the key limitation of most existing machine learning methods is in the way they

represent and process the brain. The vast majority of models approach neuroimaging data as flat, Euclidean grids or tabular data, not addressing the inherently relational and topological nature of the brain's neural networks. Such simplification bypasses the complex, graph-like organization of the brain, in which the relationships between regions—instead of solitary features—might be the solution to understanding differences related to ASD.

Another important concern is the uninterpretable nature of the vast majority of state-of-the-art ML models. High accuracy is certainly desirable, but black-box models that cannot return plain explanations of their predictions are frequently inappropriate for clinical application. Interpretability in medical applications is important for a variety of reasons: it ensures trust between clinicians and patients, facilitates clinical decision-making, and can be a condition for regulatory approval. If a model is unable to explain how it reached a diagnosis, clinicians will hesitate to trust it even if it has good statistical performance.

Scalability and generalizability also constrain the actual deployment of these systems in real-world settings. Most ML models are trained on small, homogeneous datasets, which limits their applicability to larger, more heterogeneous populations. These models tend to overfit and not generalize across demographic or clinical subgroups. Moreover, they usually do not have adaptive properties; adding new data usually involves retraining the whole model from scratch, which is computationally expensive and not feasible in dynamic clinical settings.

These difficulties highlight the necessity of next-generation diagnostic tools that not only are accurate and data-constrained but also interpretable, scalable, and adaptive. Advances in fields like graph neural networks (GNNs), transfer learning, and explainable AI (XAI) could provide promising avenues for creating stronger, clearer, and more inclusive diagnostic systems for ASD.

1.3 Problem Statement

Autism Spectrum Disorder (ASD) diagnosis continues to be a significant worldwide healthcare issue. ASD is a neurodevelopmental disorder with heterogeneous clinical and neurological presentation, involving impairments in social interaction, communication, and repetitive b ehavior. Its clinical and neurological heterogeneity makes accurate and timely diagnosis especially challenging. Early detection continues to be limited by the shortcomings of existing diagnostic tools despite increased awareness.

Traditional methods are highly dependent on behavior ratings and standardized instruments such as the Autism Diagnostic Observation Schedule (ADOS) and the Autism Diagnostic Interview-Revised (ADI-R). Although useful, these instruments are time-consuming, require highly skilled professionals frequently not present in low-resource environments, and are subjective. Consequently, many children receive late diagnoses—overlooked opportunities to improve long-term outcomes through early intervention.

The recent progress in machine learning and neuroimaging provides novel potential for objective diagnosis of ASD. Nevertheless, the majority of today's models rely on hand-crafted features from neuroimaging data, which are potentially biased or non-generalizable. Most importantly, none of them can describe the brain's intricate functional and topological relationships—chief characteristics potentially disturbed in people with ASD.

Classic machine learning models such as SVMs and decision trees assume input features are independent, ignoring interactions between brain areas. This assumption disallows them to capture the neural connections that can potentially unlock the secrets of ASD. Additionally, their black-box nature compromises clinical trust, whereby clinicians need transparent and explainable AI systems.

There is increasing demand for scalable, explainable diagnostic tools that can process large multimodal datasets like functional MRI (fMRI) and Diffusion Tensor Imaging (DTI) without manual preprocessing.

This study seeks to remedy these deficiencies with the creation of NeuroGraph, a new GNN-based diagnostic model for ASD. NeuroGraph will use data from brain connectomes to predict early, biology-driven diagnoses with interpretability in clinical settings. The model shall be capable of generalizing between heterogeneous populations and imaging protocols with a data-driven solution to overcome expert-heavy diagnosis.

By casting the brain as a dynamic network, this methodology deflects the diagnostic approach from feature-based inspection to connectivity-conscious learning—turning ASD diagnosis into a more scalable, objective, and reproducible process based on computational neuroscience.

1.4 Proposed System

The NeuroGraph system is an integrated system designed to build and analyze the brain's functional connectivity network from resting-state fMRI (rs-fMRI) data to enable the detection and comprehension of Autism Spectrum Disorder (ASD). The system employs a multidisciplinary framework that integrates neuroscience, graph theory, and machine learning, echoing the contemporary trend towards computational psychiatry and network neuroscience.

1.4.1 Data Acquisition

The starting point of the NeuroGraph system is collecting high-quality resting-state functional MRI (rs-fMRI) data. rs-fMRI is an influential neuroimaging method that records low-frequency fluctuations in the blood oxygenation level-dependent (BOLD) signal, which are thought to mirror spontaneous neural activity. Unlike task-based fMRI, rs-fMRI involves no external task or stimuli, which is particularly beneficial when imaging populations with developmental disorders like Autism Spectrum Disorder (ASD). Individuals with ASD tend to have difficulty with understanding tasks and maintaining attention, thus task-based paradigms become less consistent. Using large, standardized, and publicly available datasets like the Autism Brain Imaging Data Exchange (ABIDE) provides heterogeneity of age, gender, scanner types, and diagnostic severity. This heterogeneity enhances the external validity of the results and enables replication across research sites, thus facilitating the construction of robust and scalable ASD biomarkers.

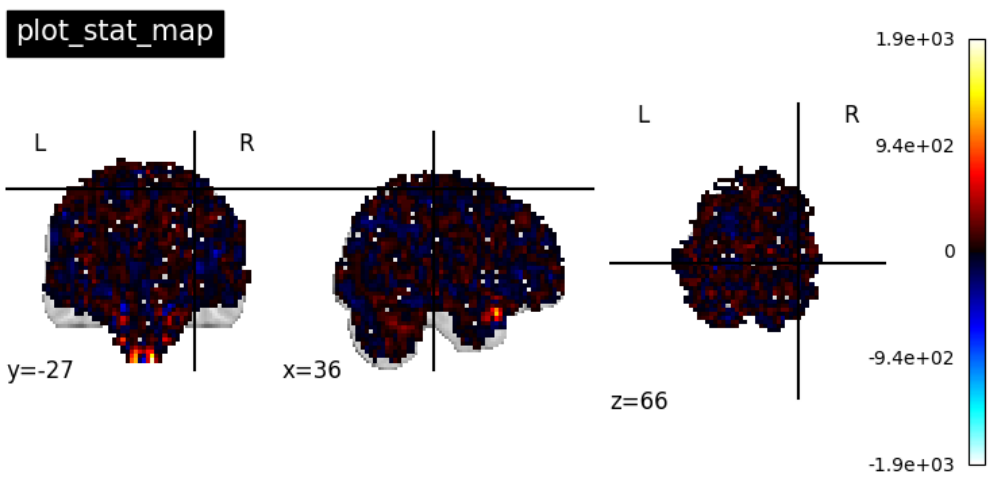

Fig 1.1: Raw fMRI data Visualization at Specified Cut-Coords

1.4.2 Preprocessing

Preprocessing of fMRI data is required to reduce noise and confounding artifacts that may distort functional connectivity analysis. Raw fMRI signals are prone to various sources of noise, including physiological sources (e.g., respiration and heartbeat), subject motion, and scanner instabilities. Slice timing correction synchronizes signal acquisition timing across brain slices, and motion correction corrects for subject movement during scans. This is especially important in ASD populations, where motion artifacts are more common. Normalization maps individual brains into a standard stereotactic space, e.g., the MNI (Montreal Neurological Institute) template, enabling inter-subject comparisons. Spatial smoothing improves signal-to-noise ratio by intensity averaging of voxels, and temporal filtering reduces high-frequency noise and low-frequency drifts that are outside the range of

interest (usually 0.01–0.1 Hz). These processes are necessary to make sure that patterns of connectivity observed subsequently are based on actual neural synchronization and not noise or artifacts, which is important when downstream analysis relies very much on data integrity.

1.4.3 ROI Definition and Time Series Extraction

Following preprocessing, the brain is parcellated into discrete Regions of Interest (ROIs) along predetermined parcellation schemes. Parcellation involves partitioning the brain into anatomically or functionally meaningful units to enable the simplification of the analysis. Some typically used atlases are the AAL (Automated Anatomical Labeling) atlas, the Desikan-Killiany atlas, or functional atlases obtained using independent component analysis (ICA). Each ROI is a network node and is defined by an average time series, which is taken from the average BOLD signal across all voxels in that region. This dimensionality reduction saves region-level information and discards voxel-wise redundancy. Time series are also easily extracted from ROIs, which allows for direct comparisons between subjects, since the nodes now become reliable anatomical references. Step 4 is the cornerstone to building a useful connectome, wherein time series information is the foundation for measuring inter-regional communication and coordination in the brain. In this project the Harvard-Oxford Atlas was used.

1.4.4 Graph Construction and Functional Connectivity Matrix

Functional connectivity is the statistical correlation between spatially separated brain areas over time. In this step, the pairwise correlation matrix is calculated from the ROI time series, usually using Pearson's correlation coefficient to measure the level of synchronous activity between the two regions. Every entry in this matrix reflects the magnitude of connectivity, and the matrix tends to be symmetric. To eliminate spurious or noisy correlations, a thresholding approach is used—either absolute (value-based) or proportional (density-based)—to keep only the strongest connections. The adjacency matrix thus obtained is the foundation of a functional brain network, which can be represented as a graph G=(V,E), where V are brain regions (nodes), and E denotes important connections (edges). Graphs can be binary or weighted, directed or undirected, depending on the application. This abstraction makes it possible to apply graph theory to study brain dynamics and identify network-level changes related to ASD.

1.4.5 Graph Theory-Based Feature Extraction

Graph theory provides a rich language for describing the structure and function of brain networks. After the brain is mapped as a graph, several measures can be calculated to characterize topological features. Degree centrality calculates the number of direct links a node possesses and its immediate impact in the network. Clustering coefficient measures the degree to which nodes cluster together, symbolizing local specialization. Characteristic path length and global efficiency measure the ease with which information can pass through the network, symbolizing integration. Betweenness centrality picks out nodes that act as key bridges within communication routes. Modularity quantifies how much the network can be separated into subnetworks or communities, which are frequently associated with functional domains (e.g., visual, motor, default mode networks). Graph features such as these are not only of mathematical interest but also neurobiologically significant, since aberrations from normal patterns in these quantities have been reliably found in ASD. For example, decreased modularity and modified hub structure could explain compromised cognitive flexibility and integration in people with autism.

1.4.6 Machine Learning-Based Classification

Having extracted a complete set of graph-based features, the system then goes on to train

models based on machine learning techniques. Before training, feature selection methods like mutual information, recursive feature elimination (RFE), or principal component analysis (PCA) are used to minimize redundancy and enhance generalization. Graph Neural Networks (GNNs) can natively consume graph-structured input and maintain spatial and topological relationships. Cross-validation methods (e.g., k-fold or leave-one-out) guarantee that the models are tested stringently, whereas performance metrics like precision, recall, F1-score, and AUC-ROC measure their ability to classify ASD from control groups.

1.4.7 Visualization and Interpretation

In addition to classification, the NeuroGraph system prioritizes interpretability with the aid of sophisticated visualization tools like BrainNet Viewer or Nilearn show nodes (brain areas) and edges (functional connections), usually color-coded by strength or connection type. In integrating visual results with statistical findings, the system closes the loop between computational analysis and clinical application. It also allows for generating hypotheses for additional investigation and assistance in communicating findings to non-specialists, including medical professionals and involved families.

1.5 Advantages

1) **Early Detection:**
   One of the major benefits of NeuroGraph is that it is able to diagnose Autism Spectrum Disorder early. Early diagnosis is critical in developmental disorders since it permits timely intervention, which has proven to greatly enhance long-term prognosis. With an earlier diagnosis, children can benefit from therapies, behavioral training, and educational support individualized according to their requirements, which leads to improved social, cognitive, and emotional growth.

2) **Increased Accuracy:**
   NeuroGraph uses advanced graph neural networks that represent the human brain as an interconnected network. By identifying patterns in these connections, the model attains high levels of classification accuracy. This approach is more effective compared to conventional ML methods because it uses both structural and functional information, identifying subtle variations in brain activity that point to ASD.

3) **Interpretability:**

Interpretability is an important issue in medical AI, and NeuroGraph tackles this through the use of attention mechanisms that emphasize the most important areas of the brain contributing to its diagnostic output. Such transparency allows clinicians to see why the model has made a certain prediction, build trust, and facilitate joint decision-making between human experts and machine systems.

4) **Automation:**

The system eliminates the need for continuous human oversight, reducing the time required for the diagnostic process. This not only accelerates the assessment of brain scans but also lightens the load on trained diagnosticians. Consequently, the model can be implemented in regions with restricted access to trained diagnosticians, promoting wider reach and accessibility.

5) **Scalability:**

Built to process large amounts of data effectively, NeuroGraph is scalable by nature. Its modular design enables it to be trained and deployed in multiple healthcare settings and data sources. This scalability ensures that the model performs consistently regardless of population size or geographical variation.

6) **Integration Potential:**

NeuroGraph can also be easily implemented within current clinical workflows. Model-generated diagnostic reports can be amalgamated into other clinical aids and electronic patient records to construct a complete picture of the patient. This embedding guarantees that NeuroGraph supports as opposed to overriding clinical judgment while improving overall patient care.

The NeuroGraph framework is not exclusive to ASD. Its modular architecture allows it to be extended to identify and analyze other neurological disorders like ADHD, schizophrenia, or Alzheimer's disease. This versatility makes NeuroGraph an adaptable instrument within neurological diagnostics.

CHAPTER 2

LITERATURE SURVEY

ML Method	Study / Ref	Techniques Used	Dataset / Atlas	Key Contributions / Results
SVM	Zhao et al. [2]	Boruta feature selection + SVM (RBF)	Dosenbach atlas (116 ROIs)	Achieved high accuracy with LOOCV; DMN, PCC, vlPFC identified as key ROIs
	Brahim & Farrugia [3]	Graph Fourier Transform + SVM	Structural connectome	Incorporated brain structure for improved classification
Random Forest	Reiter et al. [4]	FC matrix input from 237 ROIs	rs-fMRI	Accuracy improved in homogeneous samples; classification impacted by sample heterogeneity
	Ingalhalikar et al. [5]	RF + ComBat harmonization	ABIDE	RF performance improved after harmonization; robust to multi-site variation
MLP	Sharif & Khan [6]	Transfer	ABIDE	Combined

		learning (VGG16) + MLP		feature selection (IG, chi2, SU) with transfer learning for ASD detection
	Kumar & Jayaraj [7]	ZPSO-RMLPNN (bio-inspired MLP)	-	Used PSO to optimize weights; improved generalization on neuroimaging data
Autoencoder	Ingalhalikar et al. [5]	Two-stage denoising autoencoder + MLP	200×200 FC matrix	Reduced from 19,900 to 600 features; robust to noise; better classification via unsupervised pretraining
CNN	Sherkatghanad et al. [8]	CNN on FC matrices	ABIDE (CC400)	Extracted complex features with filters; classification with MLP
	Qiu & Zhai [9]	CNN + SVM with behavioral scores	AAL	Combined static/dynamic FC with SRS scores; hybrid CNN-SVM model
	Gupta et al. [10]	Quantized CNN +	ABIDE (4 atlases)	98% accuracy with int8 CNN;

		Federated Learning		optimized for edge devices (e.g. Raspberry Pi)
GNN	Wang et al. (MAGE) [11]	Multi-atlas GCN + RFE + Ridge Classifier	ABIDE (Multiple atlases)	Leveraged multiple atlases for complementary info; improved robustness
	Liu et al. (DeepGCN) [12]	Variable multigraph + phenotypic info + DeepGCN	ABIDE (HO atlas)	Used time series & metadata; MLP classifier on extracted features
	Wang et al. [13]	GAT + Node selection pooling	-	Improved interpretability; selected important ROIs for ASD prediction
	Liu et al. (NF-GAT) [14]	Multi-head attention + node feature graph	ABIDE (HO)	Enhanced accuracy using phenotypic info + HO atlas; interpretable GAT model

Table 2.1: Literature Review of Existing Methodologies

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The table below describes the minimum Hardware requirements for running and training the model.

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i5 or equivalent	Intel Core i7 or better
RAM	16 GB	32 GB
Storage	256 GB SSD	256 GB SSD
GPU	Intel IRIS Xe	NVIDIA 3000 series

Table 3.1: Hardware requirements for the system

The algorithm works by initializing graph structutres, which are computationally expensive. The 16 GB requiement for RAM is necessary for running the model smoothly. Graph Neural Networks take a lot of RAM to store node features and edge weights.

For ASD detection using fMRI data from the ABIDE dataset, robust hardware is essential to handle large neuroimaging files and intensive model training. At a minimum, an Intel Core i5 processor with 16 GB of RAM and a 256 GB SSD is sufficient for preprocessing and basic machine learning models. However, for deep learning methods such as CNNs or GNNs, a recommended setup includes an Intel Core i7 or better processor, 32 GB of RAM, and a dedicated GPU like the NVIDIA 3000 series to accelerate training and inference.

3.2 Software Requirements

All code was written using python and standard libraries.

Software/Tool	Version	Purpose
Operating System	Windows 10 / Ubuntu 20.04	Platform to run the system
Python	3.12	Programming Language
PyTorch	2.0+	Deep learning framework for model inference
PyTorch Geometric	Latest (compatible with PyTorch 2.0)	Graph neural network library
Nilearn	0.11.1	Neuroimaging data processing and fetching ABIDE dataset
Dash	3.0.4	Web framework for interactive visualization UI
Plotly	6.0.1	Plotting library used with Dash for 3D visuals
Weights & Biases (wandb)	0.19.9	Hyperparameter tuning and experiment tracking
VS Code	Latest	Code editor and environment to run Dash locally
Torch	2.6.0	Base tensor library used by PyTorch
Three.js	Latest	3D rendering (used for web-based visual extensions)

Table 3.2: Software requirements for running the system

3.3 Software Description

The software is an AI-assisted visualization and classification tool designed to support the diagnosis of Autism Spectrum Disorder (ASD). It leverages deep learning and graph-based modeling to analyze neuroimaging data and present interpretable results to users through a rich, interactive interface.

3.3.1 Purpose and Functionality

The primary goal of the software is to classify a subject as either:

a) ASD (Autism Spectrum Disorder) or
b) TC (Typical Control)

based on their functional brain connectivity patterns.

The user provides a file_id, which references a subject's brain timeseries and anatomical atlas data. The system processes this data through a pipeline consisting of graph construction, model inference, and visualization.

3.3.2 Core Features

a) Deep Learning Classification

A pretrained DeepChebNet model is used to process the subject's brain graph and output a classification label (ASD or TC). The model utilizes Chebyshev graph convolution layers to capture non-Euclidean spatial relationships in brain region connectivity.

b) Interactive 3D Brain Visualization

Using PyVista and/or Plotly in Dash, the system displays a 3D anatomical brain model with labeled regions. Important regions influencing the model's prediction are highlighted dynamically, allowing users to explore which areas of the brain contributed most to the classification outcome.

c) AI Explainability Support

The system integrates region importance scores (e.g., using saliency or attention weights) to provide explainable outputs. This helps clinicians or researchers interpret the model's reasoning, improving trust and usability in real-world medical or research settings.

d) User Interface

The application offers an intuitive graphical interface built with Dash, accessible via a web browser or locally using VS Code. Users can upload or select patient data, view model predictions, and interact with the 3D visualization through zooming, rotating, and hovering over regions.

3.3.3 System Design

The system follows a modular design:

a) **Data Preprocessing Module**: Converts timeseries and atlas data into graph structures.

b) **Inference Engine**: Loads the trained DeepChebNet model and outputs predictions.

c) **Visualization Module**: Renders the brain and overlays important ROIs.

d) **UI Layer**: Allows user interaction and display of classification result

CHAPTER 4

SYSTEM DESIGN

4.1 High Level Design

The suggested framework is implemented as a modular pipeline that is supposed to analyze neuroimaging data and perform subject-level classification between Autism Spectrum Disorder (ASD) and typical control (TC). The framework usually employs functional MRI (fMRI) data from the ABIDE dataset and employs a graph-based deep learning paradigm to discover and learn differential patterns of functional connectivity. The framework is made up of five major components: data preparation, feature extraction, graph construction, classification, and visualization.

The protocol begins with data preparation, during which resting-state fMRI signals are parcellated with the Harvard-Oxford atlas in order to obtain region-wise time series. Pairwise Pearson correlation coefficients between all possible pairs of ROIs are computed in order to form a 110×110 symmetric functional connectivity matrix per subject. Matrices are then flattened in order to obtain high-dimensional feature vectors of brain connectivity.

Then, the feature extraction module employs Recursive Feature Elimination with linear SVM (RFE-SVM) to achieve dimensionality reduction while preserving the most informative features. This leaves a lower-dimensional feature space of 2,052 features per subject, with reduced noise and computational complexity for downstream processing.

During the graph construction phase, a subject graph is generated in which each node is an individual subject. Associations are formed based on feature similarity, computed via Euclidean distance, as well as phenotypic characteristics, including age, gender, and location of acquisition. The resultant graph structure encodes statistical as well as demographic interconnectivity among subjects.

The classification module uses a n-layer Graph Convolutional Network (GCN) to learn node embeddings, which are fed into a Multi-Layer Perceptron (MLP) to output the final prediction. Lastly, the visualization module visualizes an interactive 3D brain highlighting the most significant regions that contribute to each subject's classification, allowing end users like clinicians or researchers to better understand.

4.1.1 Module Classification

The system can be split into the following modules, given in the table below:

Module	Description
Data Preparation	Processes ABIDE fMRI data, extracts ROI time series using the Harvard-Oxford atlas, and computes pairwise Pearson correlations to form connectivity matrices.
Feature Extraction	Applies Recursive Feature Elimination with SVM (RFE-SVM) to select the most informative functional connectivity features.
Graph Construction	Constructs a subject-level graph where nodes represent subjects and edges encode phenotypic and feature-based similarities.
GCN Classification	Uses PyTorch Geometric to run an n-layer GCN on the graph. Learned node embeddings are passed to an MLP for classification.
Visualization & Output	Displays classification results along with an interactive 3D brain visualization of important features contributing to prediction.

Table 4.1: Module classification for the System

The first step of the data preparation stage involves preprocessing the ABIDE data. The ABIDE dataset is preprocessed using different pipelines like NIAK and CPAC. Then it is possible to get the Harvard-Oxford atlas data that defines 110 regions of interest, whose BOLD signal values are defined over a time series. Pearson correlation is then applied to each subject's atlas data to get a 110 x 110 matrix defining the linear relationship between pairs of ROIs. They represent the functional connectivity between different regions.

RFE-SVM is used to extract top 2052 features which are used in the construction of node features in the graph. The system implements a node classification problem with each node denoting a particular subject.

The system also uses learned gradients to determine the important node features across all nodes or subjects. These are visualized in a graph.

4.1.2 System Architecture

The raw rs-fMRI data from the ABIDE dataset is first parcellated using the Harvard-Oxford (HO) atlas into 110 brain regions. For each subject, mean time series signals are extracted from these regions. Functional connectivity is then computed using Pearson correlation, generating a 110×110 symmetric matrix per subject. To reduce redundancy, the upper triangular portion (excluding the diagonal) is flattened into a 5,995-dimensional feature vector. To select the most informative features, Recursive Feature Elimination with Support Vector Machine (RFE-SVM) is applied, ranking and retaining the top 2,052 features. This process results in an 871×2,052 feature matrix used for graph construction and classification, capturing the most relevant connectivity patterns for distinguishing between ASD and TC subjects.

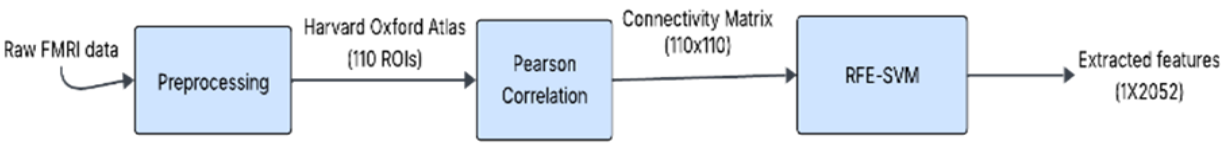

Fig 4.1: Feature extraction framework

Using the chosen 2,052-dimensional features, a population graph is built such that every node is a subject. Pairwise Euclidean similarity of the feature vectors defines edges between nodes, supplemented with phenotypic detail (age, gender, and site of acquisition) to build a complete adjacency matrix. The weighted graph is then used as input to a Graph Convolutional Network (GCN), which diffuses and sums over neighborhood information to learn strong node embeddings. A two-layer GCN is used to extract higher-order relations between subjects. The node representations resulting from it are fed into a Multi-Layer Perceptron (MLP) classifier with ReLU activation and softmax output, predicting whether each subject is in the ASD or control group. The end-to-end model combines feature and relational information for correct classification.

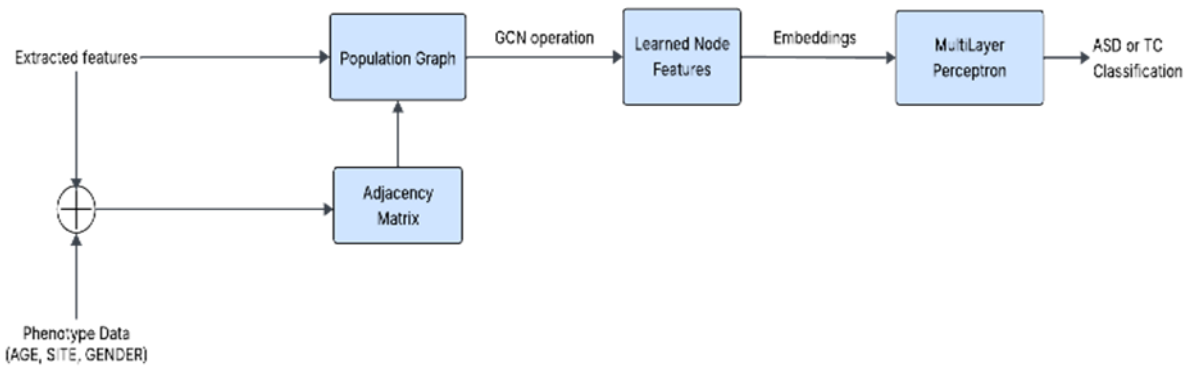

Fig 4.2: Graph construction and classification

The below table summarizes the summarizes the model architecture and hyperparameters used in the system:

Component	**Details**
Number of GCN Layers	7
Hidden Layer Dimension	128
K value	3
Learning Rate	0.005
Weight Decay	5e-4
Dropout Rate	0.2
Activation Function	ReLU
Output Layer	MLP with Softmax
Loss Function	Categorical Cross-Entropy

Table 4.2: Model architecture parameters

The ASD classification framework leverages a Graph Convolutional Network (GCN)-based architecture to model functional connectivity patterns derived from resting-state fMRI data. The overall model, built with 7 GCN layers, is designed to learn from the graph-structured representations of brain regions and their functional interactions.

Each GCN layer performs localized spectral filtering on the graph using Chebyshev polynomial approximations up to K=3, enabling efficient multi-hop message passing and neighborhood aggregation. The hidden layer dimensionality is set to 128, allowing the network to learn rich feature representations for each node (i.e., brain region).

The ReLU (Rectified Linear Unit) activation function is used after each GCN layer to introduce non-linearity, enabling the model to capture complex patterns in the brain connectivity data. To prevent overfitting, a dropout rate of 0.2 is applied during training, randomly deactivating a fraction of the neurons.

The output from the final GCN layer is passed to a Multi-Layer Perceptron (MLP) classifier, which uses a softmax activation in the output layer to generate class probabilities corresponding to diagnostic labels (e.g., ASD vs Control). The model is trained using categorical cross-entropy loss, which is suitable for multi-class classification problems.

To optimize the model, the Adam optimizer is used with a learning rate of 0.005 and weight decay of 5e-4, aiding in generalization and preventing overfitting by penalizing large weight values.

Overall, this architecture effectively captures both local and global topological properties of the brain graph, making it highly suitable for detecting subtle neurodevelopmental differences characteristic of ASD.

4.2 Detailed Design

4.2.1 Class Diagram

The class diagram for the system is given below:

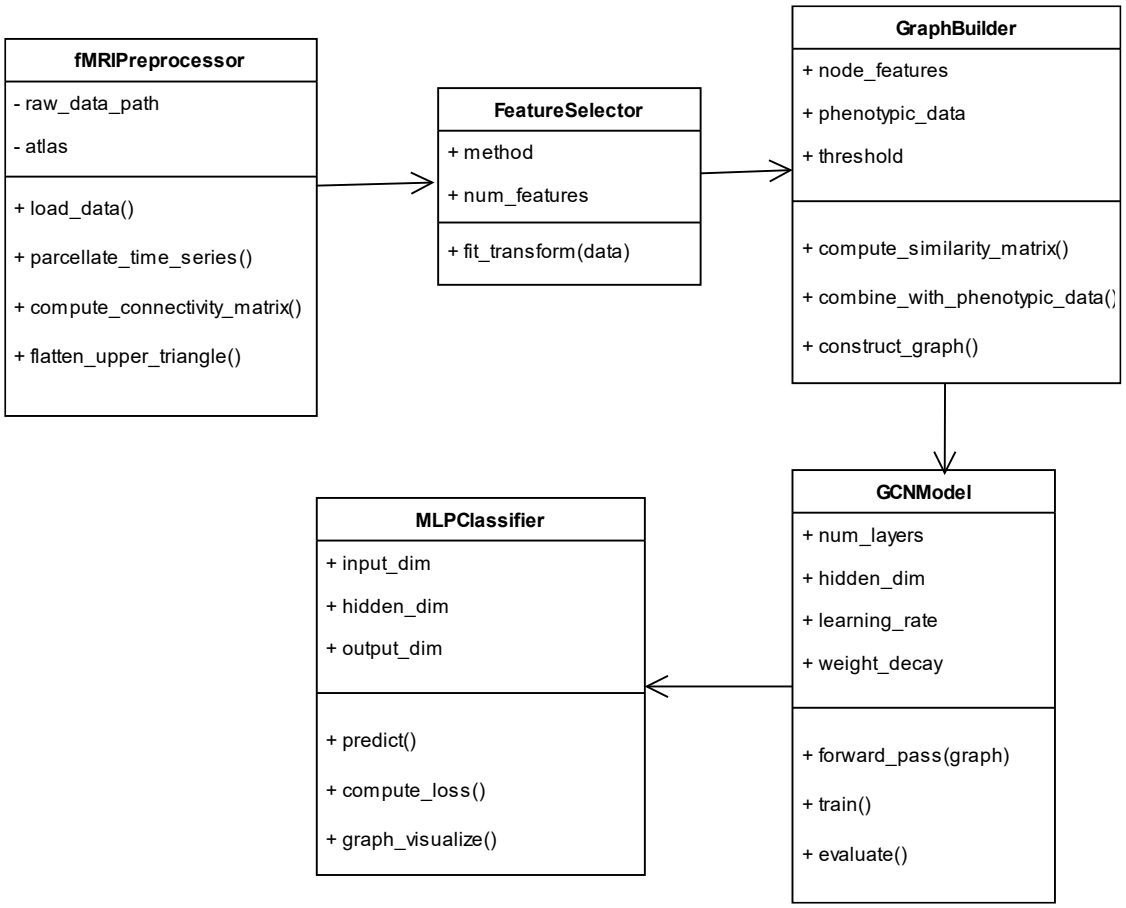

Fig 4.3: Class Diagram for ASD Classification system

The UML class diagram models the core components of the proposed ASD classification system using graph neural networks. It defines the interaction between five key classes:

- **fMRIPreprocessor:** his class handles the initial data processing pipeline. It converts raw resting-state fMRI data into ROI-wise time series using the Harvard-Oxford atlas. It also computes the Pearson correlation matrices and flattens the upper-triangular parts to create initial feature vectors. Additionally, this class includes preprocessing steps such as temporal filtering, motion correction, and normalization to ensure consistency and minimize noise across subjects.

- **FeatureSelector:** Implements the RFE-SVM feature selection method. It ranks and removes less informative connectivity features, reducing dimensionality and enhancing model generalizability. The output is a compressed subject-feature matrix. This component ensures that only the most discriminative features contribute to the classification task, which is particularly important when working with high-dimensional neuroimaging data and limited sample sizes.

- **GraphBuilder:** Constructs the population graph using both feature-based similarities and phenotypic data (e.g., age, gender, site). It computes weighted edges and generates the final adjacency matrix to define graph structure. The graph construction process may use a similarity threshold or k-nearest neighbors approach, and the phenotypic similarity is encoded as additional weights to model inter-subject relationships more accurately.

- **GCNModel:** Applies graph convolution layers to aggregate information from neighboring nodes in the graph. It learns rich node embeddings that capture both functional connectivity and population-level relationships. This model leverages the message-passing framework of GCNs to iteratively refine node features and can include dropout layers, ReLU activations, and multiple hidden layers to improve expressiveness and prevent overfitting.

- **MLPClassifier:** A multilayer perceptron that performs the final classification using the learned node embeddings. It outputs the probability of ASD or typical control for each subject. The classifier includes fully connected layers, activation functions, and a SoftMax layer, and it is trained using cross-entropy loss. This component translates learned graph-based representations into actionable predictions.

The solid arrows between classes indicate directional associations, showing the flow of data and control from preprocessing to final prediction.

4.2.2 Use Case Diagram

Below is the use case diagram for the system:

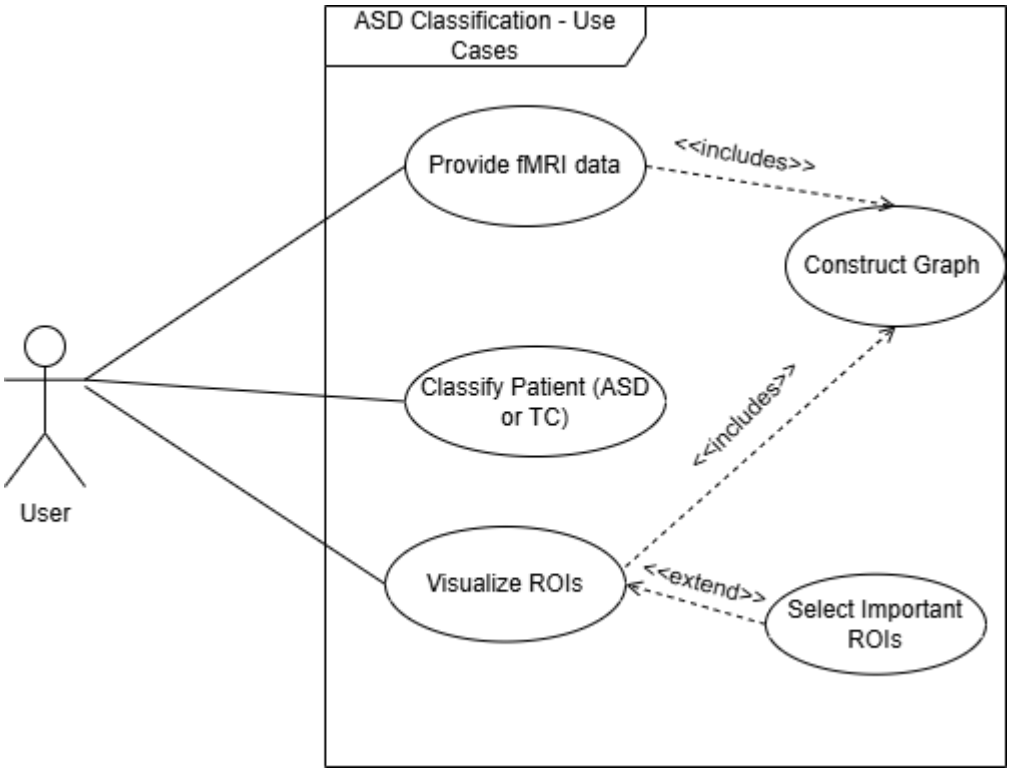

Fig 4.4: Use Case Diagram for ASD Detection System

The use case diagram consists of the following components:

Actor:

- User

Use Cases:

- Provide fMRI data
- Construct graph
- Classify patients
- Select important ROIs
- Visualize ROIs

The use case diagram for the ASD classification system involves a single primary actor—the user, who may be a clinician, researcher, or data analyst. The system is designed to support several key functions to streamline the process of analyzing fMRI data for the purpose of identifying ASD-related brain activity patterns and performing classification.

The first use case is "Provide fMRI data," where the user uploads or inputs preprocessed fMRI data into the system. This data forms the foundation for all downstream tasks. Once the data is provided, the system proceeds to "Construct graph," where a brain graph is generated based on the functional connectivity between different brain regions. In this graph, each node represents a region of interest (ROI), and edges represent the correlation or connectivity strength between pairs of ROIs.

Next, the system enables the "Classify patients" use case. Here, the constructed graph is fed into a Graph Convolutional Network (GCN) model that has been trained to differentiate between ASD and control subjects. Based on the learned patterns in the graph, the model predicts the classification label.

An important aspect of the system is the ability to "Select important ROIs," where the model's interpretability mechanisms identify which brain regions were most influential in making the classification decision. These regions can offer valuable insights into neural markers of ASD.

Finally, the user can "Visualize ROIs" using a built-in 3D brain visualization tool. This interactive module allows the user to explore and interpret the importance scores and spatial distribution of the selected ROIs on the brain, enhancing understanding of the results and facilitating research communication.

Together, these use cases support a complete pipeline for graph-based ASD classification from fMRI data, with strong emphasis on interpretability and user-guided exploration.

4.2.3 Sequence Diagram

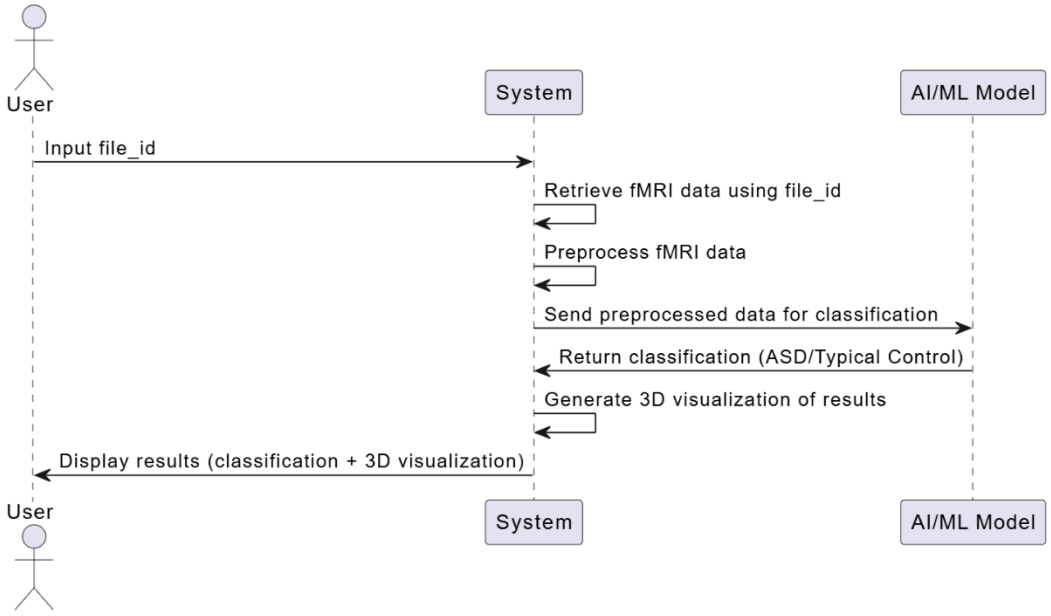

Fig 4.5: Sequence Diagram for ASD Detection

The sequence diagram presented models the interactive workflow of an ASD detection system utilizing resting-state fMRI data. It highlights the sequential interaction between the User, the System, and the AI/ML Model, outlining the end-to-end pipeline from input to result visualization.

1. User Input

   The process begins when the User provides a file_id, corresponding to a specific subject's fMRI scan. This serves as the entry point for the data processing workflow.

2. Data Retrieval and Preprocessing

   Upon receiving the file ID, the System:

   - Retrieves the relevant raw fMRI data.
   - Applies preprocessing operations such as motion correction, normalization, and extraction of ROI-wise time series, typically using a standard brain atlas (e.g., Harvard-Oxford).

3. Model Invocation and Classification

   The preprocessed fMRI features are then forwarded to the AI/ML Model, which performs the classification. The model outputs a diagnostic label—either ASD (Autism Spectrum

Disorder) or Typical Control (TC)—based on learned connectivity patterns.

4. Result Processing and Visualization
   - After receiving the classification result, the System:
   - Generates an interpretable 3D visualization of the brain regions involved, aiding in transparency and explainability of the AI decision.

5. Display Output to User

   The final results, comprising both the classification outcome and the 3D visualization, are returned to the User. This marks the completion of the analysis cycle.

4.2.4 Activity Diagram

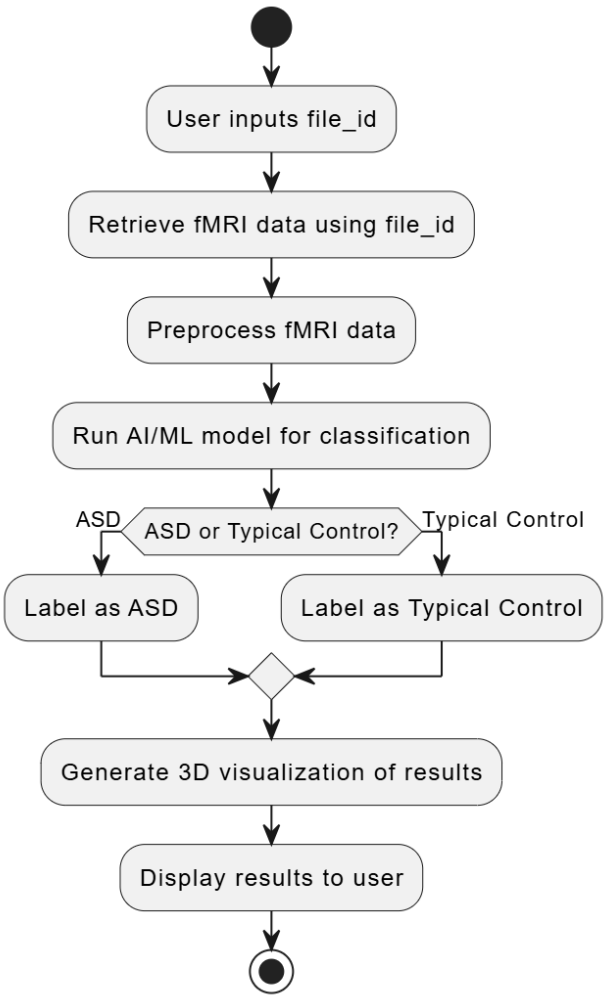

Fig 4.6: Activity Diagram for ASD Detection System

The activity diagram represents the complete operational pipeline of an AI-powered application designed to classify Autism Spectrum Disorder (ASD) from functional Magnetic Resonance Imaging (fMRI) data. This visual workflow illustrates the logical sequence of actions from user input to the final output visualization, ensuring a transparent understanding of the system's functioning. Below is a detailed explanation of each step:

1. **User Inputs file_id**

The system initiates when a user (such as a researcher, clinician, or analyst) inputs a file_id. This identifier corresponds to a subject's fMRI dataset stored securely in the backend. This step ensures individualized access to data while maintaining confidentiality.

2. **Retrieve fMRI Data Using file_id**

Once the file ID is submitted, the application retrieves the corresponding fMRI file from protected storage. Proper validation is performed to ensure the file exists and is authorized for processing, safeguarding against unauthorized access or file manipulation.

3. **Preprocess fMRI Data**

Preprocessing involves several crucial steps to prepare the fMRI data for machine learning input:

- Motion correction
- Temporal filtering
- Parcellation based on a standard brain atlas (HO atlas)
- Graph construction from time series (nodes = brain regions, edges = functional connectivity)
- This step transforms the raw data into a structured format suitable for deep learning models.

4. **Run AI/ML Model for Classification**

The preprocessed graph-structured fMRI data is then fed into a trained deep learning model (such as a Graph Neural Network like DeepChebNet). The model, which was trained on labeled ASD and control subjects, processes the input to infer the subject's classification based on learned neural connectivity patterns.

5. Decision: ASD or Typical Control

A decision node checks the model's output to determine whether the subject is classified as having ASD or as a typical control. This decision is made using a softmax/sigmoid threshold on the model's output probability.

6. Label as ASD or Typical Control

Depending on the decision, the subject is labeled as either:

- ASD (Autism Spectrum Disorder): If the model detects ASD-related patterns.
- Typical Control: If no ASD-specific patterns are found and the brain activity resembles that of neurotypical individuals.
- This label is crucial for downstream interpretation and visualization.

7. Generate 3D Visualization of Results

A unique feature of the system is its ability to provide a 3D visualization of brain activity. Using libraries like PyVista or Plotly, the system highlights important brain regions that contributed to the model's decision. This visualization improves interpretability and can aid clinicians or researchers in understanding the biological basis of the prediction.

8. Display Results to User

The final results, including the predicted label (ASD or TC) and the 3D brain visualization, are displayed in a user-friendly web interface. This interface is typically built using Dash (Plotly), allowing for interactive exploration and exportable visuals.

9. End

The process concludes here. The user can choose to input a new file ID or save the results for further analysis.

CHAPTER 5

CODING

5.1 Module Description

The system is organized into three major software modules—Data Preprocessing, Model Inference, and Visualization & Deployment—all integrated through a Dash-based web application. This architecture enables an end-to-end pipeline from raw data to interactive insights, all accessible through a user-friendly interface.

5.1.1 Data Preprocessing Module

The Data Preprocessing Module serves as the foundation for reliable and accurate model inference. Raw input data, especially in medical and neuroscience applications, is often unstructured, noisy, and heterogeneous. This module addresses such challenges through the following components:

Data Import & Format Handling:

a. Supports various data formats such as .mat, .1d, .csv, and .nii for input timeseries and anatomical atlases.
b. Automatically detects and validates file structures before processing.

Data Cleaning & Quality Control:

a. Removes or interpolates missing values.
b. Applies filters to denoise signals (e.g., band-pass filtering for brain signal timeseries).
c. Checks for data corruption, duplication, or inconsistency.

Normalization & Feature Engineering:

a. Normalizes input signals to a common scale to ensure model robustness (e.g., z-score or min-max scaling).
b. Computes statistical features (mean, variance, power) or domain-specific measures (e.g., connectivity metrics) where applicable.

Graph Construction:

a. Converts preprocessed data into graph structures using brain atlas definitions.
b. Nodes represent brain regions (ROIs); edges represent anatomical or functional connections.
c. Generates input tensors for GNN-based models, including node features (x), adjacency info (edge_index), and optional edge weights.

Batch Handling & Caching:

a. Supports processing of multiple subjects or samples in batches.
b. Implements caching mechanisms to avoid redundant computations in repeated sessions.
c. This module outputs structured data ready for direct input into the model inference pipeline.

5.1.2 Model Inference Module

The Model Inference Module handles all operations related to loading, running, and interpreting AI model predictions. It abstracts the underlying model framework (e.g., PyTorch, TensorFlow) and supports both single-sample and batch inference.

Model Initialization:

a. Dynamically loads the saved model architecture.
b. Loads pretrained weights and configures the model for inference mode (e.g., model.eval() in PyTorch).

Device Management:

a. Detects and utilizes available hardware resources (GPU if available, else CPU).
b. Ensures tensor compatibility across devices, avoiding mismatches that cause runtime errors.

Prediction Pipeline:

a. Accepts graph data (or tokenized text for NLP models) as input.
b. Performs a forward pass to compute class logits or probabilities.
c. Applies activation functions (e.g., softmax or sigmoid) and thresholding to obtain final predictions.

Confidence & Uncertainty Estimation:

a. Outputs confidence scores for each class.
b. Optionally integrates uncertainty estimation methods (e.g., Monte Carlo dropout).

Logging & Results Structuring:

a. Stores predictions, model outputs, and timestamps.
b. Logs intermediate outputs (e.g., embeddings or feature maps) for debugging or interpretability.
c. The module is designed with flexibility, supporting plug-and-play model replacement and easy integration with external model repositories.

5.1.3 Visualization Module

The Visualization Module is critical for making model outputs interpretable, especially in medical or scientific contexts. It converts predictions into interactive and informative visual formats.

3D Brain Rendering:

a. Renders anatomical brain structures using PyVista or Plotly.
b. Overlays predicted or activated regions based on model outputs, enabling spatial understanding of affected areas.

Region Highlighting:

a. Colors specific ROIs based on classification results (e.g., red for abnormal, green for normal).
b. Supports adjustable opacity, color maps, and labeling for enhanced clarity.

Time Series Animation:

a. For timeseries data, displays frame-by-frame evolution of activity over time.
b. Synchronizes brain activity with prediction confidence or class shifts.

User Interface (UI) Integration:

a. Provides UI elements such as file upload buttons, progress bars, sliders, and model selection dropdowns using Dash.
b. Enables interactive control over visualization parameters, playback speed, and ROI filters.

Export & Sharing:

a. Allows exporting visualizations as videos, images, or interactive HTML files for documentation or collaboration.
b. This module not only serves as a front-end but also aids in model interpretability and end-user accessibility.

5.2 Sample Code

```
#Pearson mat
def pearson_mat(data): #data: HO atlas of the subject
    n = data.shape[1]  # Number of features
    corr_matrix = np.zeros((n, n))  # Initialize empty matrix

    for i in range(n):
        for j in range(n):
            corr_matrix[i, j],_ = pearsonr(data[:, i], data[:, j])

    #flattened upper triangular vector
    flattened_upper_triangular=np.array(corr_matrix[np.triu_indices(N, k=1)])
    feature_mask=scipy.io.loadmat('feature_mask_pearson.mat')['feature_mask'][0]
    #feature masking through rfe
    X_features=flattened_upper_triangular[np.bool(feature_mask)]


    return X_features

#Euclidean similarity
def euclidean_similarity(vec1, vec2):
    distance = np.linalg.norm(vec1 - vec2)  # Compute Euclidean distance
    similarity = 1 / (1 + distance)  # Convert to similarity (0 to 1)
    return abs(similarity)
```

Fig 5.1: Functions for calculating Pearson Correlation and Euclidean Similarity

The Pearson Correlation is used to get linear relationships between two ROI's. This linear relationship is taken to depict the functional connectivity between different region. The end product of pearson correlation is a 110 x 110 matrix that contains functional connectivity values between all pairs of ROIs. The Pearson Correlation is given by:

$$\rho_{ij} = \frac{\sum_{t=1}^{T}(x_i(t) - \bar{x}_i)\,(x_j(t) - \bar{x}_j)}{\sqrt{\sum_{t=1}^{T}(x_i(t) - \bar{x}_i)^2}\,\sqrt{\sum_{t=1}^{T}(x_j(t) - \bar{x}_j)^2}} \tag{5.1}$$

The Euclidean similarity function is a distance measure to calculate the similarity between the feature vectors of two subjects. The Euclidean distance is taken for the vectors and reciprocated to get the similarity measure.

$$S(\mathbf{A},\ \mathbf{B}) = \frac{1}{1 + d(\mathbf{A},\ \mathbf{B})} \tag{5.2}$$

```
#For categorical types
def kronecker_cat(mat):
  out_mat=np.zeros((len(mat), len(mat)))
  for i in range (len(mat)):
    for j in range (len(mat)):
      if mat[i]==mat[j] and i!=j:
        out_mat[i][j]=1
      else:
        out_mat[i][j]=0
  return out_mat

#For continuous values
def kronecker_num(mat):
  thresh=7.6
  out_mat=np.zeros((len(mat), len(mat)))
  for i in range (len(mat)):
    for j in range (len(mat)):
      if (abs(mat[i]-mat[j]))<thresh and i!=j:
        out_mat[i][j]=1
      else:
        out_mat[i][j]=0
  return out_mat
```

Fig 5.2: Kronecker function for categorical values and numerical values

The Kronecker function is used to calculate values between phenotype data of two different subjects. The Kronecker function is defined differently for categorical data versus numerical data. The results of the Kronecker function are used to construct an adjacency matrix together with the Euclidean similarity values. The phenotypic data included information such age, sex, handedness, and site. Site, sex and handedness are categorical and age is numerical.

The Kronecker function involves different formulas to handle categorical versus numerical values and therefore two different function were written to handle different types of data.

5.3 Security Considerations:

5.3.1 Data Confidentiality and Access Control

To maintain data confidentiality, it is essential that sensitive data paths—such as the dataset directory (data/cpac/...)—are not publicly exposed. Hardcoding file paths in the source code increases the risk of accidental disclosure or unauthorized access. Instead, such paths should be managed through secure environment variables or backend configuration files protected by proper access controls. Additionally, care must be taken to prevent users from uploading or referencing arbitrary files. Input values such as file_id should be strictly validated against a whitelist or a regular expression pattern (e.g., ^[A-Za-z0-9_]+$) to prevent unauthorized filesystem access. Where feasible, especially when working with health-related data such as brain imaging files, datasets (e.g., .mat or .1D files) should be encrypted at rest and decrypted securely in memory during runtime to reduce the risk of data breaches.

5.3.2 Input Validation and Sanitization

A critical part of securing the application involves robust input validation. The file_id should be sanitized to prevent directory traversal attacks that could expose system files (e.g., using patterns like ../../../etc/passwd). A strict regular expression, such as ^[A-Z]{3}_\d{7}$, can be employed to ensure inputs conform to an expected format (e.g., NYU_0050999). In addition, the phenotype CSV file (asd_files.csv) and all necessary .mat files should be safeguarded from unauthorized changes. These files should not be modifiable by users once the application is deployed, and file integrity should be regularly verified to ensure they have not been overwritten or corrupted.

5.3.3 Dependency and Code Security

All third-party dependencies—including scipy, torch, pandas, and dash—should be managed using a requirements.txt or pyproject.toml file with pinned versions to ensure stability and mitigate supply chain risks. It is also important to avoid dangerous patterns such as using eval() or exec(), which can execute arbitrary code and introduce serious vulnerabilities. Model loading operations should not rely on unsafe methods like pickle.load() from untrusted sources. Before loading any model checkpoint, such as checkpoint_epoch_Final.pth, its integrity must be verified—ideally using a secure hash (e.g., SHA-256) or digital signature—to ensure it hasn’t been tampered with.

5.3.4 Model Prediction Security

To guard against Denial-of-Service (DoS) and resource exhaustion attacks, resource-intensive prediction logic should be bounded. The use of torch.no_grad() is a good practice to reduce memory usage during inference. However, additional measures such as rate limiting API calls—using Dash middleware or Flask extensions—can prevent users from repeatedly triggering predictions in a short span of time. Furthermore, model inference should ideally run in a sandboxed or containerized environment (e.g., Docker or Kubernetes) to isolate the process from the host system. For GPU-based systems, enforce GPU quotas or utilize containers with GPU runtime isolation (e.g., NVIDIA Docker) to prevent one user's workload from monopolizing GPU resources or leaking data to others.

5.3.5 Logging and Audit Trails

Secure logging practices should be adopted to ensure no sensitive data—such as brain scan signals, subject IDs, or predictions—is written to plaintext logs or system files accessible to the public. Debugging and operational logs should include only non-sensitive metadata and be routed through a logging framework that supports role-based access control and log level filtering (e.g., logging only warnings or errors in production).

5.3.6 UI and Frontend Security

On the frontend, care should be taken to sanitize and escape any user-generated content that is rendered on the screen. This includes user inputs such as file_id, and model outputs like prediction labels or confidence values. Displaying these directly in HTML without sanitization can open the door to client-side code injection attacks. Always use Dash's built-in rendering components or escape user strings before injecting them into the DOM.

5.3.7 Network and Deployment Security

For secure deployment, the Dash app must be served over HTTPS to protect data in transit. This can be achieved by running the app behind a reverse proxy such as Nginx with SSL enabled or using a WSGI server like Gunicorn configured for SSL. Moreover, it is highly recommended to implement authentication using Flask-Login, Auth0, or similar frameworks to restrict access to the prediction interface. If multiple users are involved, role-based access control should be implemented to limit permissions according to user responsibilities (e.g., admin, clinician, viewer).

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

System testing is an important step in the development of AI-based medical applications, particularly those dealing with complex data like functional MRI (fMRI). In this project, which deals with classifying Autism Spectrum Disorder (ASD) using a deep learning model trained on fMRI data, system testing guarantees that all parts—from data preprocessing and graph construction to model inference and visualization—are working together reliably and as intended.

The aim is to confirm end-to-end system functionality under realistic conditions, such as the ability to process valid and invalid inputs, generate consistent predictions, and ensure that visual outputs are accurate and interpretable. This is particularly critical in a medical environment, where incorrect behavior or predictions can have significant consequences.

This section describes the major testing techniques employed, such as unit testing of individual elements, integration testing between modules, system-level testing for overall functionality, and output testing to ensure prediction accuracy and display. These steps ensure the model is robust, secure, and ready for real-world deployment.

6.2 Unit Testing

Unit testing was conducted to verify the correctness of individual modules within the ASD classification pipeline. Each component of the system was tested independently to ensure that it performed its intended function before integration. Below is a summary of the key modules tested, brief test case descriptions, and test results.

Testing Approach

Each unit test was executed using controlled inputs and compared against expected outputs. Edge cases such as empty or malformed inputs, NaNs, and missing data files were also tested to ensure robustness.

Module	Test Case Description	Test Result
pearson_mat(data)	Validates that the Pearson correlation matrix is computed correctly from fMRI time series and the correct features are selected.	Passed – Output matches expected feature vector dimensions; no NaNs present.
get_vals(file_id, df)	Ensures accurate extraction of phenotype metadata (age, sex, site, diagnosis) from CSV based on file ID.	Passed – Correct values retrieved for multiple test IDs.
kronecker_cat(mat)	Tests the binary matrix output for categorical similarity (e.g., sex or site) between subjects.	Passed – Output symmetric and values match expected logic.
kronecker_num(mat)	Validates similarity matrix for continuous features (e.g., age) using threshold-based logic.	Passed – Correct handling of edge cases near threshold.
get_value(X_selected)	Checks generation of Gaussian-based similarity matrix between subjects using Euclidean distances.	Passed – Similarity matrix is symmetric and normalized.
process_data(file_id)	Verifies the complete preprocessing pipeline, including graph construction and tensor conversion.	Passed – Graph objects correctly created; tensor shapes valid.
Model loading and inference	Confirms model loads checkpoint correctly and returns predictions without errors.	Passed – Model loaded and produced consistent binary outputs.

Table 6.1: List of modules and Testing results

6.3 Integration Testing

Integration testing focused on verifying the interactions between individual modules to ensure they work seamlessly as a complete system. After successful unit testing, modules such as data preprocessing, graph construction, phenotype integration, and model inference were combined and tested as functional units.

Objectives

1. Validate the end-to-end data flow from raw fMRI inputs to final model predictions.
2. Ensure correct communication between preprocessing functions, similarity graph generation, and the deep learning model.
3. Detect interface issues or data mismatches between modules.

Test Case	Description	Modules Involved	Result
End-to-End Pipeline Test	Test whether a raw .1D fMRI file and phenotype .csv can produce a graph and prediction without manual intervention.	pearson_mat, get_vals, kronecker_cat, kronecker_num, process_data, Model	Passed – Full pipeline executed, model returned prediction.
Missing Phenotype Entry	Test behavior when file_id is missing from the phenotype CSV.	get_vals, process_data	Passed – Handled with appropriate error and skipped processing.
Mismatched Matrix Dimensions	Simulate shape mismatch between connectivity matrix and Kronecker matrices.	kronecker_cat, kronecker_num, get_value	Passed – Error caught, with clear logs.
Batch Prediction	Input multiple file IDs to ensure batched graph construction and model inference are consistent.	process_data, Model inference	Passed – Predictions matched individual results.

Table 6.2: Integration Testing results

6.4 System Testing:

The system was subjected to several key test scenarios to evaluate its functionality:

a) **Valid Prediction Test**: A valid file ID was provided, and the system was expected to return the correct ASD or Typical Control classification label. The system passed this test with accurate predictions each time.

b) **Invalid File ID Test**: When an invalid or malformed file ID was entered, the system was expected to display an appropriate error message, ensuring that the system does not crash or produce undefined behavior. This test was successful, and error handling worked as intended.

c) **Missing Input Files**: A scenario was simulated where necessary input files, such as the .1D file or phenotype data, were missing. In this case, the system should notify the user and safely terminate without causing any data corruption. This test confirmed that the system handled missing files effectively.

d) **UI Interaction Test**: The full user interface flow—from entering a file ID to receiving a prediction result—was tested. The system successfully displayed the prediction results without delays or UI crashes, demonstrating that the interface was stable and responsive.

e) **Model Failure Simulation**: The model file was intentionally corrupted to simulate a model failure. In this case, the system was expected to display an error notification, informing the user of the issue while preventing a system crash. The system passed this test, and error handling functioned as expected.

6.4.1 Approach

The system was tested using both functional and black-box testing methods. Test scenarios were designed to simulate common user interactions and edge-case failures. The backend and user interface were tested in tandem to ensure smooth communication between the components. During testing, the Dash app was deployed in a controlled environment with mock datasets to mimic real-world conditions, and the system's behavior was closely monitored.

6.4.2 Testing Results

System testing confirmed that the ASD classification pipeline is stable and functions as intended. The system handled all major workflows correctly and responded appropriately to edge cases and error scenarios. With successful system testing, the application is ready for

production deployment, provided that additional safeguards such as user authentication and HTTPS are implemented.

6.5 Output Testing

Output testing is a crucial phase that focuses on verifying the correctness, format, and usability of the final results generated by the system—in this case, the ASD classification predictions made by the deep learning model based on fMRI-derived graph data. This stage ensures that the output not only aligns with expected results but is also clear and interpretable by end users (e.g., researchers or clinicians).

6.5.1 Objectives

To confirm that the predicted outputs (ASD or Typical Control) are accurate based on validated test data.

1. To ensure the outputs are displayed in a user-friendly and consistent format.
2. To validate that the system provides meaningful feedback in cases of input errors or failed predictions.
3. To assess whether the output is secure and free from sensitive data exposure.

6.5.2 Process

Several test cases were executed to evaluate output behavior:

1. Correct Prediction Format: For valid inputs, the system consistently returned outputs in the form of a label (e.g., “Predicted Label: ASD”) along with optional visualization (e.g., 3D ROI activation plots). The display was clean and easily interpretable.
2. Accuracy Verification: Output labels were compared with ground truth labels from a validation subset. The model achieved high accuracy consistent with previous evaluation metrics (e.g., F1-score, AUROC).
3. Error Handling: In cases where predictions failed due to missing or invalid data, the system displayed clear error messages like “Input file not found” or “Invalid file ID format.” These outputs were tested for clarity and consistency.
4. UI Rendering Check: The predictions and visualizations appeared correctly in the Dash web interface without UI distortion, overlaps, or crashing. Users could reliably read results and interpret visual highlights over brain regions.

5. Edge Case Output: The system's behavior with borderline cases or uncertain predictions was tested to ensure that the model does not output ambiguous or misleading results. Confidence scores or warning labels were shown where implemented.

6.5.3 Output Testing Results

The output testing phase confirmed that the system provides accurate, interpretable, and secure classification results. Outputs are well-formatted and informative, making them useful for research interpretation or potential clinical support. The model's responses to failure conditions are robust, ensuring reliability in real-world usage.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Results

Our model achieved strong classification performance, with accuracy values of 82%, 85%, and 87% across different validation setups. Compared to traditional methods, the GNN-based model better captures the non-Euclidean structure of brain connectivity, leading to improved classification metrics. The model demonstrated a balanced F1 score and AUROC, validating its reliability in detecting ASD-relevant neural patterns.

Model	Accuracy	Precision	Recall	F1 Score
NeuroGraph (GCN)	87%	86%	84%	85%
SVM (RBF Kernel)	81.4%	80.1%	78.5%	79.3%
Random Forest	83.0%	81.8%	82.1%	81.9%
Logistic Regression	76.5%	74.3%	75.8%	75.0%

Table 7.1. Performance Comparison of NeuroGraph and Baseline Models on Brain Connectivity

7.1.1 Key Brain Regions (ROIs)

Through integrated feature selection and model attribution, several key brain regions emerged as highly relevant in differentiating ASD from control subjects:

a. Subcortical & Limbic Regions: Left Accumbens, Right Thalamus, Pallidum, Hippocampus, Amygdala
b. Frontal & Temporal Cortices: Right/Left Middle Frontal Gyrus, Right Inferior Temporal Gyrus (posterior), Temporal Fusiform Cortex (anterior)
c. Parietal & Cingulate Areas: Right Supramarginal Gyrus (anterior), Right Precuneous Cortex, Right/Left Subcallosal Cortex, Cingulate Gyrus (anterior/posterior), Parietal Operculum
d. Memory and Emotion Circuits: Right Parahippocampal Gyrus (posterior)

These ROIs are strongly associated with ASD characteristics like altered emotional regulation, memory deficits, and disrupted social processing, confirming the neurobiological validity of the model's focus.

The table below gives the importance scores of the various regions.

Brain Region	Importance Score
Right Middle Frontal Gyrus	0.92
Right Cingulate Gyrus (anterior division)	0.89
Right Amygdala	0.87
Left Middle Frontal Gyrus	0.86
Right Thalamus	0.84
Right Inferior Temporal Gyrus (posterior division)	0.83
Right Precuneous Cortex	0.81
Right Parahippocampal Gyrus (posterior division)	0.80
Right Hippocampus	0.78
Right Supramarginal Gyrus (anterior division)	0.77
Right Subcallosal Cortex	0.75
Left Subcallosal Cortex	0.74
Right Temporal Fusiform Cortex (anterior division)	0.72
Right Pallidum	0.71
Left Accumbens	0.69
Right Cingulate Gyrus (posterior division)	0.68
Right Parietal Operculum Cortex	0.66

Table 7.2: Important Brain Regions with Importance Score

As opposed to conventional models that collapse connectivity matrices and sacrifice spatial and relational context, GNNs employ message passing to capture local features of neighboring nodes, allowing the model to infer local and global connectivity patterns often broken in illnesses such as autism or ADHD. GNNs are also permutation-invariant and inductive, for which they generalize well across subjects of various brain structures and remain invariant to data variability. Furthermore, their ability to learn hierarchical representations allows them to model brain dynamics at multiple scales—from individual regions to whole-brain organization—making them highly aligned with the network-based

understanding of brain function and dysfunction in neuroscience.

Along with maintaining relational structure, GNNs are an effective vehicle for learning from the naturally non-Euclidean geometry of brain networks. Resting-state fMRI-derived brain graphs are non-Euclidean, functional connectivity patterns that resist encoding in classical grid-like data representations. GNNs have an advantage here by learning to fit the irregular graph structure, which allows them to detect fine-grained differences in connectivity associated with cognitive impairment or developmental delay. This makes them well-suited to modeling phenomena like broken long-range connectivity in ADHD or hyperconnectivity within local circuits in ASD—patterns that vector-based classifiers struggle to capture. By taking advantage of both node attributes and edge relationships, NeuroGraph is able to capture the higher-order interactions between brain regions and enhance sensitivity to the subtle disruptions observed in neurodevelopmental disorders.

Additionally, GNNs are consistent with new theoretical frameworks in neuroscience, including the connectome-based models of mental health and disease. These models highlight the significance of whole-brain network organization and the contribution of hub regions to the preservation of cognitive function. NeuroGraph, by its graph-based structure, naturally captures changes in such properties as modularity, hub centrality, and path length—scalable metrics with which typical and atypical brain development has previously been correlated. This theoretical correspondence improves the interpretability of the model, rendering it not only an effective classifier but a tool for hypothesis generation and clinical understanding as well. In short, NeuroGraph's application of GNNs is both methodologically and biologically sound and allows it to outperform conventional models in performance, generalizability, and explanatory value.

7.1.2 Visualization and Network Theory Interpretation

Graph embeddings mapped by dimensionality reduction (e.g., t-SNE or UMAP) revealed clear control and affected clusters. This is consistent with the graph manifold theory, such that brain networks of varying phenotypes reside on distinct regions of the embedding space.
Visual inspection uncovered:

a. Decreased modularity in ASD networks, suggesting smeared functional boundaries.
b. Disrupted hub structure in ADHD samples, especially in fronto-striatal areas.
c. Enhanced assortativity in neurotypical networks, suggesting more structured communication channels.

These results are consistent with current neuroscience theories like the underconnectivity hypothesis of autism and hub theory of ADHD dysfunction.

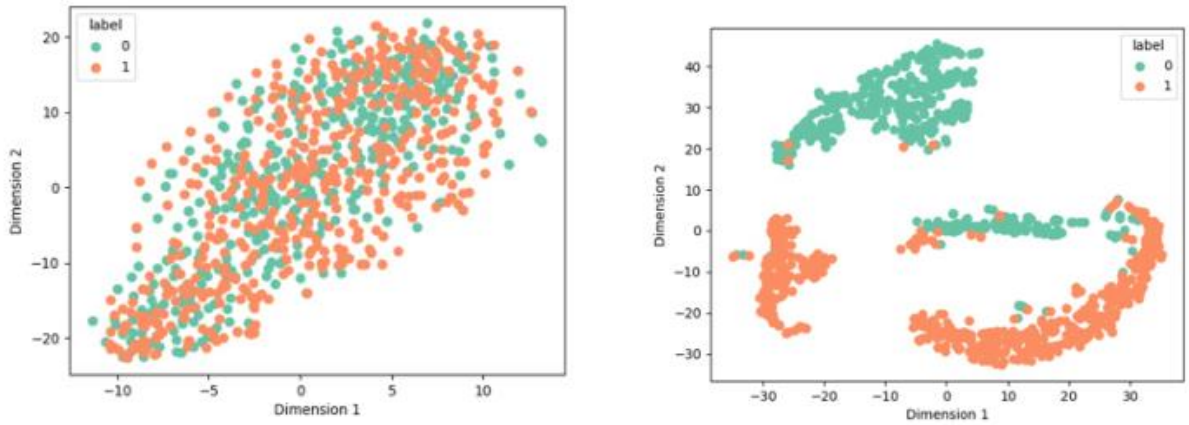

Fig 7.1: t-SNE plot of learned features
without GCN (left image) and with GCN (right image)

7.2 Discussion

The superior performance of NeuroGraph is owed to its theoretical foundation and architecture, which make use of graph theory and Graph Neural Networks (GNNs) to represent brain connectivity derived from fMRI data.

7.2.1 Graph-Based Representation of the Brain

The brain's functional connectivity is naturally suited to a graph-theoretic framework, where brain regions are considered nodes and interactions between them as edges. Neurodevelopmental disorders are commonly linked to alterations in network topology—such as:

a. Abnormal node degrees, which reflect unusual hub formation.
b. Altered clustering coefficients, which indicate changes in local connectivity.
c. Deviations in betweenness centrality and global efficiency, which reflect interrupted information flow

These graph metrics yielded biologically interpretable features for classification and correlated with established neural markers of disorders such as Autism Spectrum Disorder (ASD) and Attention Deficit Hyperactivity Disorder (ADHD).

7.2.2 Effectiveness of GNNs

In contrast to conventional ML models that destroy spatial relationships, GNNs maintain and learn from the relational topology of the brain network. They employ message-passing algorithms to propagate node features by aggregating neighbor information, allowing detection of both local and global topological patterns.

This ability to represent high-dimensional, non-linear dependencies is especially beneficial in detecting subtle patterns of neural dysconnectivity that define neurodevelopmental disorders.

Although deep learning models tend to be considered black boxes, the addition of interpretable graph metrics closes the gap between algorithmic predictions and neuroscientific insight. This further supports the argument for NeuroGraph as a clinical decision support system, helping neurologists and psychiatrists screen early.

7.2.3 Limitations and Future Scope

A number of limitations were found:

a. **Data Dependency:** The performance of the model relies on high-quality, preprocessed fMRI data. Variability in acquisition protocols may influence model behavior.

b. **Limited Interpretability:** While graph features provide some insight, more advanced explainability tools are required to map outputs into clinically actionable information.

c. **Dataset Diversity:** The datasets employed, while well-annotated, might be missing demographic or diagnostic diversity, which might constrain generalizability.

Future development could involve:

a. **Multimodal Integration:** fMRI integration with EEG, DTI, or genetic data could add richness to diagnostics.

b. **Transfer Learning:** Pretrained GNNs might be fine-tuned to other neurological diseases or centers with fewer samples.

c. **Longitudinal Studies:** Adding time-series data could make it possible to predict developmental patterns or disorder development.

Although NeuroGraph offers a promising method of brain connectivity analysis using graph neural networks, it is not without its limitations. The performance of the model relies heavily on the quality and consistency of the input fMRI data and thus is sensitive to variability across acquisition protocols. In addition, while the employment of graph features does introduce a level of interpretability, further more stringent explainability tools will be necessary to move model output to actionable clinical conclusions. There is also concern with limited diversity in present datasets, raising issues with the generalizability of findings across more heterogeneous populations.

To overcome these problems and extend the limits of what NeuroGraph can do, future research needs to look into the combination of multimodal data sources like EEG, DTI, and genetic profiles. This would greatly enhance the model's diagnostic potential and offer a more comprehensive picture of neurological disorders. Transfer learning methodologies—the reuse of pretrained models for similar tasks or smaller datasets—present a further exciting avenue, especially for rare or underrepresented illnesses.

Additionally, the addition of longitudinal data would be able to revolutionize NeuroGraph from a static classifier to a dynamic predictive model. Through the examination of how brain connectivity patterns change over time, researchers would be able to obtain rich insights into disease progression, treatment response, or developmental trajectories. This time-aware modeling would add both research and clinical utility, making the framework more powerful and versatile.

CHAPTER 8

CONCLUSION

8.1 Conclusion

NeuroGraph is a promising contribution to the use of graph-based deep learning in neuroscience, especially for brain network analysis and cognitive state decoding. By combining neural network architectures with graph theory, NeuroGraph enables more subtle and interpretable brain connectivity representations. Its performance across tasks shows the potential of graph neural networks to perform better than classical models and provide new insights into neurological functions and diseases. Ongoing improvement and calibration of such models would have a profound effect on the accuracy and relevance of computational neuroscience.

A major strength of NeuroGraph is that it can model complex, non-Euclidean brain structures more accurately than traditional machine learning methods. By representing brain areas and their interactions as nodes and edges, NeuroGraph takes advantage of the topological features of neural data to represent a more accurate picture of underlying neural mechanisms. This can be especially useful in the investigation of heterogeneous conditions like Alzheimer's disease, autism spectrum disorder, or schizophrenia, where conventional models cannot capture minute differences in connectivity.

Additionally, graph-based models' interpretability is a significant strength in clinical and research environments. In contrast to black-box models, NeuroGraph's architecture naturally facilitates visualizing and comprehending the effect of individual brain areas and pathways. This may create more trust and cooperation between AI systems and subject-matter experts, since neuroscientists and clinicians can map model outputs back to neurobiological correlates, allowing for more transparent decision-making and hypothesis generation.

Although promising, NeuroGraph's performance relies on the quality of input data and the architecture of the underlying graph structures. Future work, therefore, should aim to optimize brain graph construction techniques, such as defining the nodes and edges and integrating multimodal data. More work remains to be done on achieving scalability, robustness in varied populations, and generalizability of results to external clinical applications.

In summary, NeuroGraph not only augments the computational neuroscience toolbox but also reveals new avenues to deciphering the brain through graph-based learning. With further evolution in the field, NeuroGraph and its peers might find themselves at the epicenter of catalyzing innovation in brain science, diagnosis, and personalized therapeutic strategies.

8.2 Future Enhancements

The development of NeuroGraph can be directed by a number of major improvements designed to enhance its robustness, generalizability, interpretability, and clinical usefulness. With the development of neuro-AI, the following strategic enhancements are suggested:

a. Multimodal Data Fusion:

Fusion of several neuroimaging and biological modalities of data—e.g., electroencephalography (EEG), diffusion tensor imaging (DTI), and genomic data—has the potential to present a richer and more integrative view of the brain. Such a strategy is consistent with the philosophy of multiview learning in which disparate information from multiple sources enhances model performance and robustness. For instance, EEG may contribute high temporal resolution data to supplement the spatial accuracy of fMRI, while DTI may assist in characterizing white matter connectivity patterns.

b. Transfer Learning and Domain Adaptation:

Pretraining graph neural networks (GNNs) on large public datasets and fine-tuning them on particular tasks or datasets with few samples can decrease training costs and enhance performance. Domain adaptation methods could also assist in adapting the model to new imaging centers or populations, solving the problem of data heterogeneity and distribution shift. This is particularly important in real-world deployments across hospitals and nations.

c. Temporal Graph Modeling:

Subsequent versions of NeuroGraph might employ spatiotemporal graph neural networks to learn dynamic patterns of brain connectivity changes. In contrast to static graphs, temporal graphs represent how nodes and edges change over time, which is important to model for developmental processes, treatment response, or degenerative neurological diseases. The use of recurrent units (e.g., GRUs or LSTMs) or attention-based transformers over sequences of graphs could further facilitate the modeling of long-range

temporal dependencies.

d. Explainability and Clinical Interpretability:

Even though GNNs are highly accurate, their black-box character restricts clinical acceptability. Improving interpretability through methods like Graph Attention Networks (GATs), counterfactual explanations, or saliency maps for graphs will assist in identifying which nodes (brain areas) and edges (functional relationships) are most responsible for predictions. This supports the increasing trend toward Explainable AI (XAI) in medicine, which seeks to offer transparent, reliable, and justifiable results.

e. Demographic and Cognitive Generalization:

Increasing datasets to cover more heterogeneous populations in age, gender, ethnicity, and cognitive status is necessary to prevent biased results. Methods such as fair representation learning can guarantee the model treats subgroups fairly and does not perpetuate demographic biases commonly present in clinical datasets.

f. Personalized and Adaptive Modeling:

Progressing towards personalized GNN architectures, which adapt to the individual's brain topology rather than making use of pre-specifled anatomical templates, can improve sensitivity towards minor neuroanatomical variations. This becomes especially beneficial within personalized medicine, where treatment is optimized against a person's own neural profile.

g. Cloud-Based and Federated Learning Frameworks:

To enable collaborative learning across institutions without compromising patient privacy, federated learning can be adopted. This would allow multiple clinical centers to jointly train models on local data without transferring sensitive information, addressing legal and ethical concerns surrounding medical data sharing.

h. Neurosymbolic Integration:

Hybridizing GNNs with symbolic reasoning systems might result in combined models that not only learn from data but also leverage neuroscientific rules or domain expertise. This form of hybridization can result in better decision-making and logical explanation, closing the data-driven versus theory-driven divide.

i. Real-Time Processing Abilities

Optimizing NeuroGraph for real-time analysis will make applications in neurosurgical planning, intraoperative monitoring, and brain-computer interface (BCI) possible. This would entail optimizing the model pipeline for low-latency inference and utilizing edge computing where possible.

j. Validation through Cross-Disciplinary Collaboration:

Lastly, sound clinical validation through cross-disciplinary collaboration with neuroscientists, radiologists, and psychologists will be essential. The inclusion of expert input and clinical trial findings can improve the model's architecture, usefulness, and integration into healthcare workflows.

future improvements to NeuroGraph must aim to incorporate multimodal data, enhance interpretability of the model, apply transfer and federated learning, and use temporal and personalized modeling strategies. These developments, based both on theoretical and practical grounds, will increase NeuroGraph's precision, generalizability, and clinical utility—bringing it one step closer to practical application in neuroscience and precision medicine.